

MUSIC RECORDS STORE - VITYARTHI PROJECT REPORT

Submitted by : - Palak Kalra

Registration number : - 26BCY10001

Course : - Introduction to problem solving and programming

Platform : - VITYarthi, GITHUB

Introduction

The Records Store Management System is a python based project designed to manage a music records store. It provides separate menus for the owner, employees, and customers. The system supports inventory management, employee records, customer accounts, record searching, shopping cart operations, checkout, and basic financial management. It demonstrates fundamental Python programming concepts.

Problem Statement

Problem : -

Managing a records store manually can make it difficult to keep track of available stock, employee details, customer purchases, and store finances. As a result, there is no centralized data system that everyone can access based on their needs.

Solution : -

I have created a menu-based Python code that organizes all store data, like employee information, consumer purchases, employee and customer accounts, and financials, in Python data structures, which makes it easy to keep track of and no errors that aren't handled by the code.

Functional Requirements

The project was created using core Python data structures and have not used any external modules. User-defined functions for handling problems like viewing the inventory, adding to it, deleting records from it, editing the records, searching for records according to their name, artist, year, and price range, adding to cart, deleting from it, checking out, viewing employee information, editing employee information, adding a new employee, deleting an old employee from the database, paying salaries, checking the current money held by the store, and editing it were created by me.

Owner functions (all rights) : Viewing, Adding, Deleting, and Editing employees information; managing inventory; all rights to access the store's money to check and view it; calculating and paying salaries.

Employees functions : View inventory, Add to inventory, Delete from inventory, and Edit inventory

Customers functions : View inventory, Search for record by name, Search for record by artist, Search for record by year released, Search for record according to price range, Add to Cart, Delete from cart, and Check Out.

Non-Functional Requirements

Usability: Text-based menus and numbered options guide users through the program.

Access Control: Different menus are provided for each user role, with up to three login attempts.

Maintainability: Functions separate major tasks such as searching, editing, and checkout.

Portability: The program uses standard Python features and runs in a Python 3 environment.

Data Limitation: Data is held in memory; changes are not retained after the program closes.

System Architecture

This project follows a simple modular, function based structure.

- Console displays menus and prompts and results using print() and input()
- Functions perform authentication, searching, cart updates, inventory management, employee management, and salary calculations.
- Lists and dictionaries store inventory, cart contents, login details, employee information, and the store's money during execution.

Diagrams

Use Case Diagram

Owner use cases: Manage employees, manage inventory, view/update money, pay salaries.

Employee use cases: View and manage inventory.

Customer use cases: Register/login, browse and search records, manage cart, checkout.

Workflow Diagram

Start

|

V

Choose User Type

|

+---- Owner -----> Login -----> Owner Menu

|

+---- Employee ---> Login -----> Employee Menu

|

+---- Customer ---> Login/Create -> Customer Menu -> Shopping

Functions -> Checkout

Design

Lists: Used for inventory rows and cart records, where each record contains name, artist, price, release year, and quantity.

Dictionaries: Used to associate usernames or employee IDs with passwords and employee details.

Functions: Used to divide the program into manageable operations and reduce repeated code.

Console Interface: A text-based interface keeps the project simple and focuses on core programming logic.

Functions used

Function	Purpose
add_to_cart()	Adds a record to the customer's cart
delete_cart()	Removes records from the cart
search_name()	Searches for a record by name
search_artist()	Searches for records by artist
search_year()	Searches for records by release year
search_price()	Searches for records within a price range
view_cart()	Displays the customer's cart and subtotal
check_out()	Completes the customer's order
view_inventory()	Displays all inventory
add_employee()	Adds a new employee
delete_employee()	Deletes an employee
edit_employee_info()	Changes employee information
view_employee_info()	Displays employee information
add_inventory()	Adds a new record to inventory

<code>delete_inventory()</code>	Deletes a record from inventory
<code>edit_inventory()</code>	Changes record information
<code>check_money()</code>	Displays the store's available money
<code>edit_money()</code>	Changes the store's available money
<code>pay_salary()</code>	Pays all employee salaries

Challenges Faced

Maintaining consistency between the cart quantity and inventory quantity, handling different permissions and menus for three user types, updating nested lists and dictionary values correctly, and checking user choices and quantities to avoid invalid operations

Learnings and Key Takeaways

- Practised defining and calling Python functions.
- Learned to store and change values using lists and dictionaries.
- Learned loops and conditionals by implementing them in menu's, authentication, searching, and validation.
- Understood how to represent inventory, cart totals, and salary calculations as code.
- Recognized the importance of testing edge cases and breaking code into smaller functions.

Future enhancements

- Use secure password handling instead of storing passwords directly in the source code.
- Add receipts, order history, and a graphical user interface.
- Connect to a payment gateway for an actual check out.
- Improve database management and input handling

References

- Project source code: Records Store Management System (user-provided Python program).
- Python Tutorial – Data Structures:
<https://docs.python.org/3/tutorial/datastructures.html>
- Python Documentation: <https://docs.python.org/3/>
- Module PPTs

